

- TD 1 -

Prise en main ou révisions de Python

Exercice 1 : *Comprendre les commandes usuelles du type `list`*

Taper dans la console Python les instructions ci-dessous.

Chaque passage à la ligne sera accompagné de la touche « return » du clavier : la ligne de commande est exécutée et la console affiche le résultat demandé le cas échéant. Prendre le temps d'observer le résultat des commandes.

```
L=[1,2,3]
L
len(L)
L.append(0)
L[-1]
L
L.insert(3,"a")
L
u=L.pop(3)
u
L
L=L+[2]*3
L
L[0]=5
L
L.count(2)
L.count(8)
L.index(2)
L.index(3)
L.index(0)
2 in L
10 in L
```

Exercice 2 : *Comprendre les commandes usuelles du type `dict`*

1. Taper dans la console Python les instructions ci-dessous.

Chaque passage à la ligne sera accompagné de la touche « return » du clavier : la ligne de commande est exécutée et la console affiche le résultat demandé le cas échéant. Prendre le temps d'observer le résultat des commandes.

```

D={1:[0,1,2,3],2:[], "G3":[0,1]}
D
D[1]
D["G3"]
D[4]="ajout"
D
del D[4]
D
D[2]=D[1]
D
D[2]=3*4
D

```

2. Dans l'éditeur, taper les commandes suivantes :

```

D={"E1":[75,90,100,"non note"],\
  "E2":[60,85,65,75],\
  "E3":[65,"absent",70,85],\
  "E4":[50,60,70,80]}
# on peut imaginer que D est le relevé de notes (sur 100)
# de 4 étudiants, nommés ici E1, E2, E3 et E4
s=0
for k in D.keys():
    L=D[k]
    print(L)
    if 75 in L:
        s=s+1
print('s= ',s)

```

Observer ce qu'affiche la console après exécution.

Adapter le script précédent pour qu'il affiche le nombre de devoirs sans notes, ainsi que le nom des étudiants ayant été absents à un devoir.

Exercice 3 : Savoir définir des matrices de taille n

Pour cet exercice, on pensera à charger le module `numpy`, via la commande : `import numpy as np`. Une fois vos fonctions Python écrites, vous pouvez vérifier qu'elles sont correctes, en les exécutant pour de petites valeurs de n .

1. Soit $n \in \mathbb{N}^*$ et

$$A_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ \frac{1}{2} & \frac{1}{2} & 0 & \dots & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & \ddots & \vdots \\ \vdots & & & \ddots & 0 \\ \frac{1}{n+1} & \frac{1}{n+1} & \dots & \frac{1}{n+1} & \frac{1}{n+1} \end{pmatrix} \in \mathcal{M}_{n+1}(\mathbb{R}).$$

Ecrire une fonction en Python, d'en-tête `def Mat(n):`, de paramètre `n` entier strictement positif, utilisant une unique boucle `for` et qui renvoie la matrice A_n ci-dessus.

2. Soit $n \in \mathbb{N}^*$ et

$$M_n = \begin{pmatrix} 0 & 1 & 0 & \dots & \dots & 0 \\ 1 & 0 & 1 & \ddots & & \vdots \\ 0 & 1 & \ddots & \ddots & & \vdots \\ \vdots & \ddots & \ddots & & \ddots & 0 \\ \vdots & & \ddots & & \ddots & 0 \\ \vdots & & & \ddots & \ddots & 1 \\ 0 & \dots & & 0 & 0 & 1 & 0 \end{pmatrix} \in \mathcal{M}_n(\mathbb{R}).$$

Autrement dit : $M_n = (m_{i,j})_{1 \leq i,j \leq n}$ avec, pour tout $(i,j) \in \llbracket 1, n \rrbracket^2$, $m_{i,j} = \begin{cases} 1 & \text{si } |i-j| = 1 \\ 0 & \text{sinon} \end{cases}$.

Écrire une fonction en Python, d'en-tête `def Mat2(n):`, utilisant une unique boucle `for` et qui renvoie la matrice M_n ci-dessus.

3. Soit $n \in \mathbb{N}^*$. Soit $x = (x_1, \dots, x_n) \in \mathbb{R}^n$ et $M_x = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{pmatrix} \in \mathcal{M}_{n,2}(\mathbb{R})$.

Écrire une fonction en Python d'en-tête `def matM(x):`, où `x` est de type `array` et représente le vecteur x de $\mathbb{R}^n$, afin qu'elle renvoie la matrice M_x .

Exercice 4 : Manipulation de `if ... then ... else`, `for` et `while`

Pour cet exercice, on pensera à charger le module `math`, via la commande : `import math as m`.

1. Ecrire une fonction nommée `sol`, de paramètres `b` et `c` de type `float`, qui renvoie le nombre de solutions réelles de l'équation du second degré $x^2 + bx + c = 0$, d'inconnue x .
2. Ecrire une fonction Python, de paramètre `n` entier strictement positif et `a` réel, qui renvoie le terme u_n de la suite $(u_n)_{n \in \mathbb{N}}$ définie par :

$$u_0 = a, \quad \text{et} \quad \forall n \in \mathbb{N}, \quad u_{n+1} = \frac{e^{u_n}}{u_n^2 + 1}$$

3. On considère la suite $(u_n)_{n \in \mathbb{N}}$ définie par :

$$u_0 = a \quad \text{et} \quad \forall n \in \mathbb{N}, \quad u_{n+1} = u_n \cos(nu_n).$$

Compléter les deux fonctions ci-dessous afin qu'elles renvoient la valeur du terme u_n de la suite $(u_n)_{n \in \mathbb{N}}$.

```
def suite1(a,n):
    u=a
    for k in range(n):
        ???
    return(u)
```

```
def suite2(a,n):
    u=a
    for k in range(1,n+1):
        ???
    return(u)
```

4. Écrire une fonction en Python, d'en-tête `def f(x,n):`, de paramètres x réel et n entier naturel, qui calcule la valeur de $\sum_{k=0}^n \frac{x^k}{k!}$.
On essaiera de faire le moins de calculs possibles en Python.

5. Écrire un script en Python qui calcule et affiche le plus petit entier $n \geq 1$ tel que $\left| \sum_{k=1}^n \frac{1}{k^2} - \frac{\pi^2}{6} \right| \leq 10^{-3}$.

Exercice 5 : Manipulation de listes

1. Les instructions en Python :

```
n=20
L=list(range(0,n,2))
```

crée la liste des entiers pairs de 0 à $n - 1$, rangés par ordre croissant.

Compléter la fonction Python ci-dessous, de paramètre n entier strictement positif, afin qu'elle renvoie la même liste L obtenue à l'aide d'une boucle `for`.

```
def listepaire(n):
    L= ???
    k= ???
    for i in range(k+1):
        L=L+[2*i]
    return(L)
```

2. Écrire une fonction `echange` de paramètres L de type `list`, et i et j de type `int`, afin qu'elle renvoie une liste de même composantes que L , sauf les termes de L situés aux rangs i et j qui ont été permutés.

Après avoir écrit votre fonction, vous pouvez l'utiliser dans la console pour vérifier qu'elle fonctionne. Vous pourrez par exemple exécuter les commandes ci-dessous, chaque passage à la ligne étant accompagné de l'appui de la touche « return » du clavier :

```
L=list(range(11))
L
M=echange(L,2,4)
M
```

3. Lorsque L est une variable de type `list` à coefficients réels, la commande Python `max(L)` renvoie la valeur du maximum parmi les composantes de L .
Compléter la fonction Python ci-dessous afin qu'elle renvoie également le maximum de la liste L , le paramètre de la fonction.

```
def maxlist(L):
    m=L[0]
    n= ???
    for k in range(1,n):
```

```

        if ??? :
            m= ???
    return(m)

```

4. Ecrire une fonction Python d'en-tête `def ordre(L):`, de paramètre `L` de type `list` à coefficients tous réels, qui renvoie `True` si les éléments de `L` sont rangés par ordre croissant, et qui renvoie `False` sinon.
- 5.
6. On considère la fonction Python ci-dessous, de paramètres `L` de type `list` et `x` de type quelconque.

```

def rangs(L,x):
    if not(x in L):
        return('non dans la liste')
    else:
        M=L.copy()
        # crée une variable Python, nommée M, initialement égale à L
        k=M.index(x)
        r=[k]
        M.pop(k)
        i=1
        while x in M:
            k=M.index(x)
            r=r+[k+i]
            M.pop(k)
            i=i+1
        return(r)

```

- (a) Après avoir recopié la fonction `rangs` dans l'éditeur, l'utiliser dans la console pour la liste `L=[1,2,1,1,1,2,3,1]` et le réel 10, puis 3, puis 2, puis 1.
Que renvoie la fonction `rangs`?
Quelle est l'utilité de la variable Python `i`?
- (b) Écrire une fonction Python `rangsbis` de même paramètres et de même utilité que la fonction `rangs`, sans utiliser la commande `pop`, mais utilisant des sous-listes de `L` de la forme `L[i:j]`.

Exercice 6 : Manipulation de dictionnaires

1. Écrire une fonction Python de paramètre `n`, représentant un entier strictement positif, et qui renvoie le dictionnaire de clés `1, 2, ..., n` et dont la valeur associée à la clé `i`, pour $i \in \llbracket 1, n \rrbracket$, est la liste `[0, 1, 2, ..., i]`.
2. On suppose que `KY3` est un dictionnaire listant les groupes de khôlles de votre promotion :
 - chaque clé est de type `int` et correspond au numéro d'un des groupes de khôlle de votre promotion en 2025-2026,

- pour chaque clé, la valeur correspondante est la liste des prénoms des étudiants du groupe de khôlle correspondant (de type `str`).

On supposera que tous les étudiants de votre promotion ont des prénoms distincts.

- Quelle instruction Python renvoie le nombre de groupe de khôlles de votre promotion ?
- Compléter les instructions ci-dessous afin d'afficher dans la console le numéro du groupe de khôlle de l'étudiant dont le nom a été entré au clavier.

```
Nom=input('entrer votre nom')
```

- Écrire une suite d'instruction en Python permettant d'afficher le nombre de groupes de khôlles composés de deux étudiants.
- Écrire une procédure qui, pour un prénom donné, renvoie `Non` en Y3 si l'étudiant n'est pas dans votre promotion, et qui renvoie `Oui, dans le groupe i`, si l'étudiant est bien dans votre promotion et dans groupe de khôlle n° *i*.

Vous pourrez tester le fonctionnement de vos procédures sur un exemple fictif, en utilisant par exemple le dictionnaire :

```
KY3={1: ["Léna", "Kevin", "Anne"], \
      2: ["Ava", "Léa", "Héloïse"], 3: ["Iris", "Loïc", "Rapahël"], \
      4: ["Gabrielle", "Luc"]}
```

- Inventer un dictionnaire avec une signification concrète, comme le dictionnaire KY3 de la question précédente. Puis utiliser ce dictionnaire pour résoudre différents problèmes liés aux données décrites dans ce dernier.

Exercice 7 : Un exercice concret sur les dictionnaires

- Télécharger le fichier `countries.xlsx`. Recopier ensuite les lignes de commande suivante (en adaptant le chemin d'accès au fichier `countries.xlsx`) qui permettent de créer un dictionnaire dont les clés sont les différents pays du monde, et pour chaque clé, la valeur associée est la capitale du pays concerné.

```
import pandas
fichier = pandas.read_excel('C:/Users/DELL/Documents/
    SPEIT_Graphes/TD_graphes_25-26/countries.xlsx', header=None
    )
Pays=fichier[0]
# Pays est la première colonne du fichier excel, constituée
  de la liste de tous les pays
Capitales=fichier[1]
# Capitales est la seconde colonne du fichier excel,
  constituée de la liste des capitales correspondantes

Dico={}
n=len(Pays)
for i in range(n):
    Dico[Pays[i]]=Capitales[i]
```

2. Écrire un programme dans lequel l'ordinateur choisit aléatoirement un nom de pays et vous fait deviner sa capitale. En cas de succès, il affiche **Gagné !**, et affiche si non : **Perdu ! La bonne réponse était ***** .

On utilisera les commandes suivantes pour sélectionner un des pays au hasard dans la liste des clés du dictionnaire :

```
import random as rd
P=rd.choice(list(Dico.keys()))
```

3. Modifier le programme précédent pour qu'il compte 1 point par succès et affiche votre score final au bout de cinq parties.
4. Modifier le programme précédent pour qu'il vous autorise trois essais à chaque partie.
